



Tecnicatura Universitaria
en Programación

PROGRAMACIÓN IV

Unidad Temática N°2:
Microservicios

Material de Estudio
2° Año – 4° Cuatrimestre



Contenido

Introducción y objetivos	2
¿Qué es un microservicio?	2
Autónomos	4
Especializados	5
Monolitos vs Microservicios	6
Escalabilidad	7
Ventajas y desventajas de la arquitectura de microservicios	9
Ventajas	9
Desventajas	10
Tolerancia a fallos y gestión de errores.	11
Tolerancia a fallos:	11
Gestión de errores:	13
Resiliencia	14
Patrones y Buenas Prácticas en microservicios	15
Circuit Breaker	17
API Gateway	20
Health Check	23
Rate Limit	23
BFF (Backend for Frontend)	24
Event-Driven Architecture (EDA)	27
Service Discovery	29
SAGA	32
Modos de SAGA	33
Coreografía	34
SIDECAR	34
Domain-Driven Design (DDD)	35
Domain (dominio)	36
Lenguaje Ubicuo	36
Bounded Context	37

Introducción y objetivos

La arquitectura de **microservicios** se popularizó en la última década por su capacidad de responder a los límites del monolito y a la exigencia de entregar software con mayor frecuencia, calidad y autonomía de equipos. En lugar de concentrar todo en una única base de código y despliegue, este enfoque propone **dividir por capacidades de negocio**, de modo que cada parte pueda evolucionar a su propio ritmo, escalar de forma independiente y fallar sin arrastrar a todo el sistema.

El objetivo de esta unidad es que el estudiante **comprenda qué es (y qué no es)** un microservicio, cuándo conviene adoptarlo y cómo trazar **caminos de migración realistas** desde un monolito saludable hacia una arquitectura distribuida. El foco está en las **decisiones de diseño** (límite de servicio, datos privados, contratos) y en los **costes operativos** que trae lo distribuido (observabilidad, despliegues, seguridad y resiliencia). La comunicación entre servicios puede ser **síncrona** (REST o gRPC) o **asíncrona** (eventos/mensajes); privilegiar esta última reduce el acoplamiento temporal y mejora la escalabilidad organizativa.

¿Qué es un microservicio?

Los microservicios son una **arquitectura de software** que ha ganado una gran popularidad en la última década debido a su capacidad para abordar los desafíos asociados con sistemas monolíticos y el creciente enfoque en la entrega rápida y eficiente de aplicaciones. Esta arquitectura ofrece una manera moderna y escalable de desarrollar, desplegar y mantener aplicaciones complejas, brindando una mayor flexibilidad y eficiencia para las organizaciones y equipos de desarrollo.

En el contexto de la arquitectura de microservicios, una aplicación completa se divide en un conjunto de servicios más pequeños e independientes que se conocen como “microservicios”. Cada uno de estos microservicios representa una **funcionalidad de negocio específica y bien delimitada** dentro de la aplicación general—idealmente alineada a un **bounded context** de *Domain-Driven Design* (DDD)—lo que permite un enfoque modular y un desacoplamiento entre las distintas partes del sistema. Cada microservicio es responsable de una tarea o una función del negocio y opera de manera independiente, lo que facilita su desarrollo, implementación, escalabilidad y mantenimiento.

La principal premisa detrás de los microservicios es la división del sistema en componentes más manejables y especializados, en lugar de tener un gran monolito donde todas las funcionalidades están fuertemente acopladas. Esto permite que cada

microservicio se desarrolle y se despliegue de manera independiente, sin afectar a los demás. De hecho, diferentes equipos de desarrollo pueden trabajar en microservicios separados al mismo tiempo, lo que fomenta la colaboración y aumenta la velocidad de desarrollo. **Cada servicio, además, controla sus propios datos (*database per service*)**, lo cual evita dependencias ocultas y facilita la independencia real de cambio.

La comunicación entre los microservicios se realiza generalmente a través de interfaces bien definidas, como API (Interfaces de Programación de Aplicaciones) **RESTful o gRPC, y también mediante eventos o mensajería asíncrona**. Estas interfaces permiten que los microservicios se comuniquen entre sí de manera eficiente y estandarizada. Al tener una interfaz clara y definida para cada microservicio—con **contratos explícitos y versionados** para mantener compatibilidad hacia atrás—se facilita la comprensión de cómo interactúan los diferentes componentes y se mejora la mantenibilidad y escalabilidad del sistema.

Además, los microservicios permiten que los equipos utilicen diferentes tecnologías y lenguajes de programación para desarrollar cada servicio según las necesidades específicas. Esto proporciona una gran flexibilidad y evita que un equipo quede limitado a una única tecnología para toda la aplicación, siempre dentro de una **poliglotía controlada** que respete estándares transversales (seguridad, observabilidad, empaquetado y despliegue).

La arquitectura de microservicios también facilita la **escalabilidad horizontal**, lo que significa que se pueden agregar más instancias de un microservicio específico para manejar un mayor número de solicitudes o cargas de trabajo, sin afectar el rendimiento global del sistema. Esto se debe a que cada microservicio es autónomo y puede escalar por separado, lo que permite una mayor eficiencia en el uso de recursos.

Sin embargo, a pesar de sus numerosos beneficios, los microservicios también presentan desafíos significativos, especialmente en términos de **gestión y monitoreo/observabilidad** de múltiples servicios, así como en el manejo de la complejidad de las interacciones entre ellos. La **orquestación**, el **monitoreo/observabilidad (trazas, métricas y logs)**, la **seguridad** y la **consistencia de datos** son aspectos críticos que deben considerarse cuidadosamente al adoptar esta arquitectura.

Entonces... ¿Qué son los microservicios?

Es un **enfoque arquitectónico y organizativo** para el desarrollo de software en el que el sistema está compuesto por **pequeños servicios autónomos y especializados** que se comunican a través de **APIs bien definidas y eventos/mensajes**. Organizativamente, los propietarios de estos servicios son **equipos pequeños e independientes**. **Técnicamente, cada servicio controla sus propios datos (*database per service*) y sus límites surgen del dominio** (bounded context), lo que habilita evolución y despliegue independientes con contratos claros y versionados.

Autónomos

Cada servicio componente en una arquitectura de microservicios se puede desarrollar, implementar, operar y escalar sin afectar el funcionamiento de otros servicios. Los servicios no necesitan compartir ninguno de sus códigos o implementaciones con otros servicios. Cualquier comunicación entre componentes individuales ocurre a través de API, eventos o mensajes. Algunas características clave que hacen que los microservicios sean autónomos incluyen:

- **Despliegue independiente**: Cada microservicio puede ser implementado y actualizado por separado sin afectar a los otros servicios. Esto permite una mayor agilidad en el despliegue de nuevas funcionalidades o correcciones de errores, ya que no es necesario detener o recompilar todo el sistema para realizar un cambio en un microservicio específico.
- **Lógica de negocio específica**: Cada microservicio está diseñado para abordar una tarea o función específica del negocio. Esto significa que cada servicio se enfoca en una sola responsabilidad, lo que facilita el mantenimiento y la comprensión del código.
- **Datos propios (*database per service*)**: Cada microservicio controla su propio almacenamiento y modelo de datos. No comparte tablas ni bases de datos con otros servicios; la integración entre dominios se realiza por contratos externos (APIs o eventos), no mediante accesos directos a la base de datos de otro servicio.
- **Comunicación a través de interfaces**: Los microservicios se comunican entre sí a través de interfaces claramente definidas, como API RESTful o gRPC, **y también mediante eventos o mensajería**. Esto proporciona un acoplamiento mínimo y evita que los servicios se entrelacen en exceso.

- **Escalabilidad independiente:** Los microservicios pueden escalar de manera independiente según su demanda específica. Si un servicio experimenta una carga alta, se pueden agregar más instancias de ese servicio sin afectar la escalabilidad de otros.
- **Tecnología y plataforma independiente:** Cada microservicio puede estar desarrollado en diferentes tecnologías o lenguajes de programación, lo que brinda a los equipos de desarrollo la libertad de utilizar las herramientas más adecuadas para cada servicio.
- **Aislamiento de fallos:** La autonomía de los microservicios permite que un fallo en uno de ellos no afecte directamente a los demás. Esto ayuda a mantener la estabilidad general del sistema y a aislar y resolver problemas de manera más efectiva.

Especializados

Cada servicio está diseñado para un conjunto de capacidades y se enfoca en resolver un problema específico **alineado a una capacidad de negocio dentro de un *bounded context* (DDD)**. Si los desarrolladores aportan más código a un servicio a lo largo del tiempo y el servicio se vuelve más complejo, se puede dividir en servicios más pequeños **(sin caer en “nanoservicios” y siempre respetando el límite de dominio)**. La especialización de un microservicio tiene varias implicaciones importantes:

- **Enfoque en una sola responsabilidad:** Cada microservicio tiene una funcionalidad limitada y claramente definida, lo que facilita el entendimiento de su propósito y el mantenimiento del código asociado. Esto mejora la legibilidad y facilita la resolución de problemas y actualizaciones.
- **Desarrollo y despliegue ágil:** Al ser específico en su propósito, el desarrollo de un microservicio se puede llevar a cabo de manera más rápida y eficiente. Además, su despliegue e implementación son independientes, lo que agiliza el proceso de lanzamiento y actualización.
- **Acoplamiento reducido:** Al tener funcionalidades especializadas y bien definidas, los microservicios minimizan las dependencias entre sí. Esto permite una mayor flexibilidad y facilidad para modificar o mejorar un servicio sin afectar a otros.

- **Escalabilidad dirigida:** Los microservicios especializados permiten escalar solo aquellos componentes que enfrentan una mayor carga de trabajo, sin tener que escalar todo el sistema. Esto brinda una mejor utilización de recursos y una mayor eficiencia.
- **Mejor adaptación a la evolución del negocio:** Al tener una especialización clara, los microservicios son más adaptables a los cambios en los requerimientos o en el enfoque del negocio. Pueden ser modificados o reemplazados sin afectar al resto del sistema.
- **Límites correctos de dominio (evitar capas técnicas):** La especialización debe definirse por **dominio/capacidad de negocio**, no por capas técnicas (p. ej., “service de controladores” o “service de repositorios”). Dividir por capas reintroduce acoplamientos y complica la evolución.
- **Contratos y datos propios con versionado:** La especialización se sostiene con **contratos explícitos y versionados** (APIs/eventos) y **datos/esquemas privados** (*database per service*). Esto permite evolucionar sin romper a consumidores y mantener la independencia real de cambio.

Monolitos vs Microservicios

Con las arquitecturas monolíticas, todos los procesos están estrechamente asociados y se ejecutan como un solo servicio o artefacto de despliegue. Esto implica que, si un módulo de la aplicación experimenta un pico de demanda, la forma usual de escalar es **replicar todo el artefacto** (escala horizontal) o aumentar los recursos de la instancia (escala vertical); **no es posible escalar ese módulo en forma aislada**. Agregar o mejorar características se vuelve más complejo a medida que crece la base de código, lo que limita la experimentación y dificulta la implementación de nuevas ideas. Las arquitecturas monolíticas también pueden aumentar el riesgo de disponibilidad porque el alto grado de dependencia interna eleva el impacto de un fallo no contenido.

Con una arquitectura de microservicios, una aplicación se crea con componentes independientes que implementan **capacidades de negocio** dentro de un **bounded context**, ejecutando cada capacidad como un servicio. Estos servicios se comunican a través de una interfaz bien definida mediante **APIs ligeras** (REST/gRPC) **y/o eventos/mensajería**, se diseñan para **una sola función de negocio** y **controlan sus propios datos** (*database per service*). Debido a que se ejecutan de forma independiente, cada servicio se puede actualizar, implementar y escalar para satisfacer la demanda de funciones específicas de una aplicación **sin impactar al resto**.

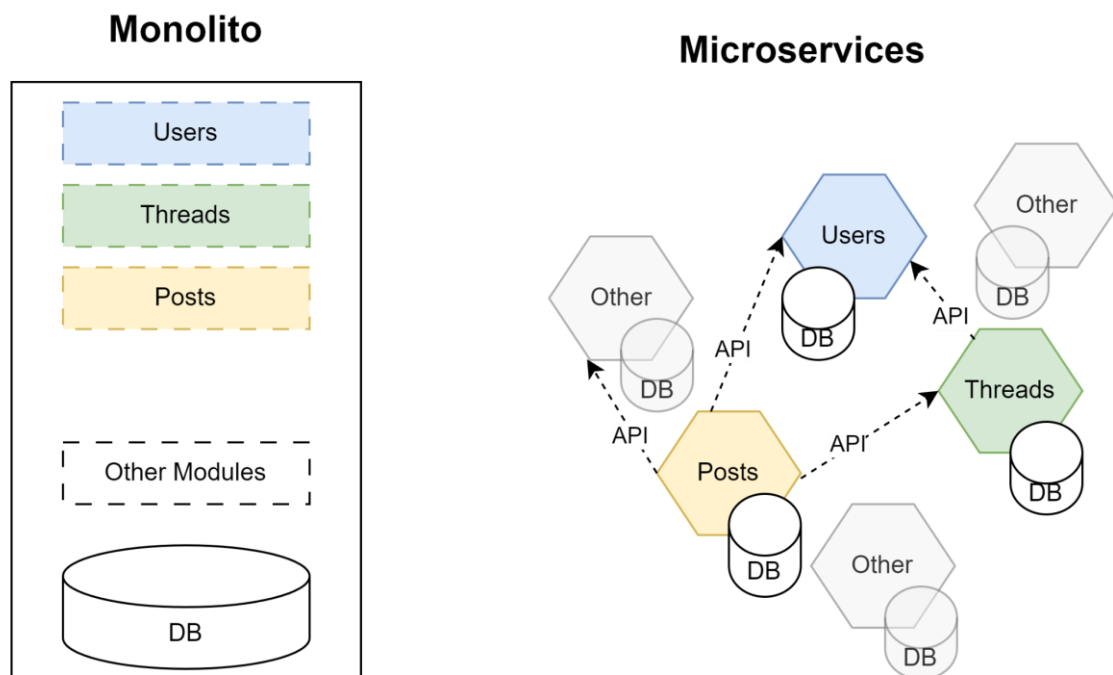


Imagen 1: Elaboración Propia.

Escalabilidad

La **escalabilidad** —término **reconocido por la RAE**—, en el contexto de las arquitecturas de software y sistemas, se refiere a la capacidad de un sistema para adaptarse y manejar eficientemente un aumento en la demanda de recursos o cargas de trabajo sin perder rendimiento o funcionalidad. Es decir, un sistema escalable es capaz de crecer y adaptarse de manera flexible y eficiente para satisfacer las necesidades cambiantes y crecientes de los usuarios o aplicaciones.

La escalabilidad es un aspecto crucial para garantizar que una aplicación o sistema pueda manejar el crecimiento y la expansión sin degradar su rendimiento ni causar interrupciones en el servicio. La capacidad de escalar es esencial en un mundo digital en constante evolución, donde la demanda de servicios y aplicaciones puede variar significativamente a lo largo del tiempo.

Existen dos tipos principales de escalabilidad en el contexto de sistemas de software:

Escalabilidad Vertical (escalado hacia arriba): Este enfoque consiste en aumentar los recursos (como CPU, memoria RAM o almacenamiento) en un solo servidor o máquina para mejorar su rendimiento y capacidad. Por ejemplo, se puede mejorar un servidor agregando más memoria o cambiando a un procesador más potente. Sin embargo, el escalado vertical tiene un límite práctico, y alcanzar ese límite puede volverse costoso o incluso técnicamente inviable.

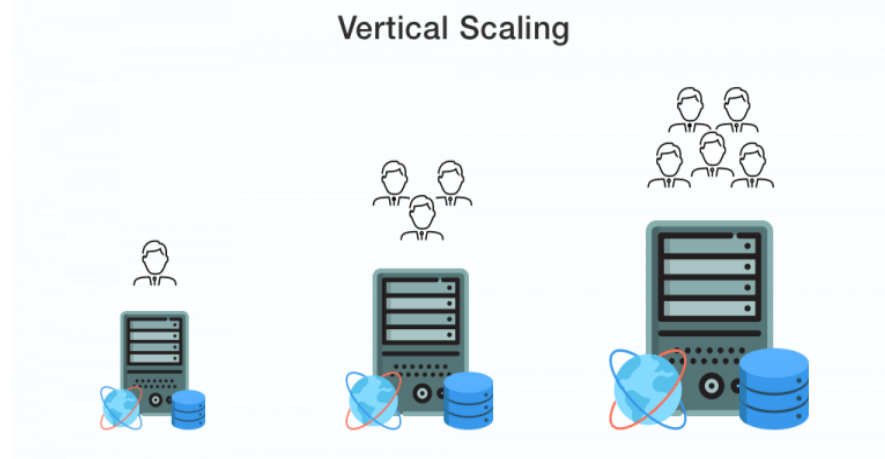


Imagen 2: Elaboración Propia.

Escalabilidad Horizontal (escalado hacia afuera): En este enfoque, se aumenta la capacidad del sistema agregando más instancias o servidores idénticos para distribuir la carga de trabajo. En lugar de mejorar un solo servidor, se agregan más servidores que trabajan de manera conjunta para manejar la demanda creciente. El escalado horizontal es más flexible y puede brindar una mayor escalabilidad, ya que permite agregar recursos adicionales a medida que crece la demanda.

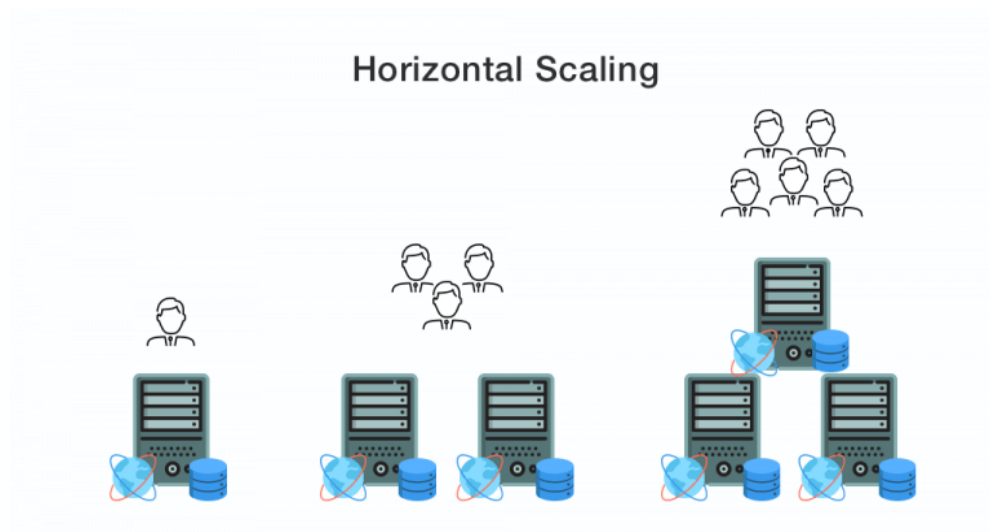


Imagen 3: Elaboración Propia.

Además de estas formas, es útil distinguir la **elasticidad**, es decir, la capacidad de **aumentar y disminuir automáticamente** la cantidad de recursos en respuesta a la demanda (por ejemplo, horaria o estacional). La elasticidad suele apoyarse en **mecanismos de autoescalado** basados en métricas (CPU, memoria, RPS, latencia) y en **umbrales alineados a SLO/SLI** para no sobrereaccionar ni degradar la experiencia.

Para que la escalabilidad horizontal sea efectiva, los componentes **sin estado (stateless)** resultan más sencillos de replicar; cuando hay **estado** (sesiones, cachés locales o bases de datos), se requieren estrategias específicas como **almacenamiento compartido, replicación, particionamiento (sharding)** o **uso de cachés/colas externas**, evitando que el estado quede “pegado” a una instancia concreta.

En el contexto de las arquitecturas monolíticas y de microservicios, los microservicios tienen la capacidad de crecer horizontalmente de manera **dirigida a la demanda**: pueden escalar **solo** los servicios que lo necesitan, acompañando picos de consumo y reduciendo capacidad cuando el tráfico desciende. Esta granularidad permite una utilización de recursos más eficiente y un control más fino de costos y rendimiento.

Ventajas y desventajas de la arquitectura de microservicios

Las arquitecturas de microservicios ofrecen una serie de ventajas y desventajas que deben considerarse cuidadosamente al adoptar este enfoque en el desarrollo de software.

A continuación, se presentan algunas de las principales ventajas y desventajas de las arquitecturas de microservicios:

Ventajas

- **Desarrollo ágil y despliegue independiente:** Los microservicios permiten el desarrollo y despliegue independiente de cada componente, lo que acelera el proceso de desarrollo y mejora la agilidad del equipo.
- **Escalabilidad y flexibilidad:** La escalabilidad horizontal de los microservicios permite agregar o quitar instancias de servicios específicos según la demanda, lo que brinda una mayor flexibilidad y eficiencia en el uso de recursos.

- **Mantenibilidad simplificada:** Al tener servicios más pequeños y especializados, el mantenimiento y las actualizaciones se pueden realizar de manera más enfocada y sin afectar a otros componentes del sistema.
- **Independencia tecnológica (poliglotía controlada):** Cada microservicio puede estar desarrollado utilizando diferentes tecnologías, lo que brinda a los equipos la libertad de elegir las herramientas más adecuadas para cada tarea, manteniendo estándares transversales de seguridad, observabilidad y despliegue.
- **Alineación equipo–servicio y trabajo distribuido:** Los equipos pueden trabajar en microservicios separados, con **ownership** claro por servicio, lo que favorece la colaboración entre equipos distribuidos geográficamente.
- **Resistencia a fallos:** Los microservicios pueden diseñarse para aislar fallos y evitar que afecten a todo el sistema.
- **Evolución segura del contrato:** La existencia de **APIs/eventos con contratos explícitos y versionados** facilita cambios graduales con compatibilidad hacia atrás, reduciendo fricción entre equipos.

Desventajas

- **Complejidad en la gestión y coordinación:** Con múltiples servicios independientes, la gestión y la coordinación entre ellos puede volverse compleja, especialmente en sistemas grandes.
- **Sobrecarga de comunicación:** La comunicación entre microservicios a través de redes introduce **latencia** y **sobrecarga** adicionales, especialmente si no se diseña ni se cachea adecuadamente.
- **Posible degradación del rendimiento:** Una mala planificación de la comunicación entre microservicios puede llevar a una degradación del rendimiento general del sistema.
- **Mayor esfuerzo de pruebas y observabilidad:** Las pruebas de **contrato e integración** y la instrumentación de **trazas, métricas y logs** requieren más disciplina debido a la naturaleza distribuida del sistema.
- **Consistencia de datos y transacciones distribuidas:** Mantener la consistencia a través de múltiples servicios es un desafío; suele implicar **consistencia eventual**, **SAGA** y, cuando aplica, patrones como **Outbox/CDC** para publicación confiable de eventos.

- **Resiliencia no “gratis”:** Se necesitan **timeouts**, **reintentos con backoff + jitter**, **circuit breakers**, **idempotencia** y **rate limiting** para evitar fallas en cascada.
- **Seguridad distribuida:** Aumenta la superficie de ataque y la complejidad de **autenticación/autorización** entre servicios (p. ej., **OIDC/mTLS**, **Zero Trust**).
- **Requerimientos de plataforma y coste operativo:** El despliegue de múltiples microservicios exige **CI/CD**, **service discovery**, **API gateway**, **gestión de configuraciones y secretos**, y a veces **service mesh**; todo ello incrementa el costo y la operación.

Tolerancia a fallos y gestión de errores.

La tolerancia a fallos y la gestión de errores son aspectos críticos en el diseño y desarrollo de sistemas, especialmente en entornos distribuidos como las arquitecturas de microservicios. Estos conceptos se refieren a cómo un sistema maneja situaciones inesperadas o problemas en su funcionamiento para asegurar la continuidad del servicio y la recuperación ante fallos. A continuación, se detallan ambos aspectos.

Tolerancia a fallos:

La tolerancia a fallos es la capacidad de un sistema para continuar funcionando de manera adecuada y proporcionar una funcionalidad degradada o una recuperación automatizada en caso de que uno o varios de sus componentes (como microservicios) experimenten fallos o errores.

En una arquitectura de microservicios, la tolerancia a fallos implica diseñar los microservicios y su interacción de tal manera que el fallo en uno de ellos no afecte negativamente a todo el sistema. Algunas prácticas comunes para lograr la tolerancia a fallos son:

- **Redundancia y replicación:** Mantener copias redundantes de los microservicios y sus datos en diferentes nodos o zonas/regiones (activo–activo o activo–pasivo), de modo que si una instancia falla, otra pueda tomar su lugar sin interrupción significativa.

- **Circuit Breaker (interruptor de circuito):** Implementar el patrón de *circuit breaker* para detectar errores o latencias anómalas y **abrir** el circuito, evitando nuevas solicitudes hacia el servicio problemático hasta que se recupere (estado **half-open** para probar la recuperación).
- **Respuestas degradadas (*fallbacks*):** En lugar de fallar por completo, los microservicios pueden proporcionar respuestas degradadas (por ejemplo, datos en caché, valores por defecto o supresión temporal de una funcionalidad no crítica).
- **Gestión de colas y reintentos:** Si un microservicio no responde temporalmente, se puede reintentar la solicitud **con política de *exponential backoff* + *jitter* y límite de intentos**, preferentemente encolando mensajes. **Solo reintentar operaciones idempotentes**; para el resto, usar **transacciones compensatorias (SAGA)** o marcar para revisión humana. Utilizar **Dead-Letter Queue (DLQ)** para mensajes que superen el umbral de reintentos.
- **Manejo adecuado de excepciones:** Capturar y manejar excepciones de forma consistente para evitar propagación descontrolada, **clasificando** entre errores transitorios (reintentables) y permanentes (no reintentables), y entre **errores técnicos (5xx)** y **errores de negocio/validación (4xx)**.
- **Timeouts y *deadlines* en clientes:** Definir **timeouts explícitos** y presupuestos de tiempo (*request deadlines*) en todas las llamadas remotas para evitar esperas indefinidas y cola de conexiones bloqueadas.
- **Aislamiento de recursos (*bulkheads*):** Separar *pools* de conexiones/hilos/memoria por dependencia para impedir que una dependencia lenta o caída **arrastre** al servicio completo.
- **Limitación de carga y *load shedding*:** Aplicar **rate limiting**, cuotas y **rechazo temprano** cuando el sistema está saturado, protegiendo la latencia de las operaciones críticas y preservando la estabilidad.
- **Healthchecks y autorecuperación:** Exponer *startup/liveness/readiness probes* y usar *grace periods* y *circuit breakers* internos para permitir reinicios controlados, *drain* de conexiones y rotación de instancias sin afectar usuarios.

Gestión de errores:

La gestión de errores se refiere a la forma en que el sistema maneja los errores y las situaciones excepcionales cuando ocurren. En una arquitectura de microservicios, es fundamental implementar una gestión de errores robusta para proporcionar información significativa sobre los fallos y facilitar la resolución de problemas. Algunas prácticas para la gestión de errores incluyen:

- **Registro y observabilidad de errores:** Registrar y monitorear los errores en los microservicios para identificar patrones y tendencias de fallos y facilitar su resolución, incorporando **trazas, métricas y logs correlacionados** (por ejemplo, con un **trace-id** común).
- **Respuestas con códigos de estado adecuados:** Devolver **códigos HTTP** o códigos de error significativos para que los clientes entiendan la naturaleza del problema y respondan en consecuencia. Diferenciar claramente **4xx** (errores del cliente/validación/negocio) de **5xx** (errores del servidor/técnicos). Incluir un **cuerpo de error estructurado** (código interno, mensaje legible, **traceId**, y, cuando aplique, **retry-after** para **429/503**), evitando exponer detalles sensibles.
- **Notificación y alertas:** Configurar sistemas de notificación y alertas para que el equipo de desarrollo sea informado inmediatamente cuando ocurra un error crítico. Basar las alertas en **SLO/SLI** (tasa de errores, latencia, saturación), con **deduplicación** y niveles de severidad para reducir ruido.
- **Reintentos y recuperación:** Implementar mecanismos de reintentos para errores **transitorios**, usando **exponential backoff + jitter** y un **límite máximo de intentos**. **Reintentar solo operaciones idempotentes**; para las no idempotentes, aplicar **compensaciones (SAGA)** o intervención manual. En flujos asíncronos, utilizar **colas** y **Dead-Letter Queue (DLQ)** para mensajes que exceden el umbral de reintentos.
- **Respuestas amigables para el usuario:** Proporcionar mensajes de error claros y comprensibles para los usuarios finales, evitando mensajes técnicos o poco informativos. No exponer *stack traces* ni datos sensibles; registrar esos detalles de forma interna.
- **Tiempos de espera y deadlines (añadido):** Definir **timeouts** explícitos en clientes/servidores y **presupuestos de tiempo** por solicitud para evitar esperas indefinidas y liberar recursos oportunamente.

- **Clasificación de errores (añadido):** Distinguir **errores transitorios** (reintentables) de **permanentes** (no reintentables) y **errores de negocio** de **errores técnicos**; esta taxonomía guía políticas de reintento, *fallbacks* y el mapeo a códigos de estado.
- **Contratos tolerantes y versionados (añadido):** Diseñar **APIs/eventos** con versionado y patrón **tolerant reader** para que los consumidores manejen campos desconocidos sin fallar, reduciendo errores por cambios de contrato.
- **Seguridad en el manejo de errores (añadido):** **Sanitizar** mensajes y *logs* para no filtrar PII/secretos, y usar **correlación** (p. ej., *traceld*) para diagnóstico sin exponer internals al cliente.

Resiliencia

La resiliencia, en el contexto de los sistemas informáticos y las arquitecturas de software, se refiere a la capacidad de un sistema para resistir, recuperarse y adaptarse de manera efectiva ante situaciones inesperadas, fallos, errores o condiciones adversas. Un sistema resiliente es capaz de mantener su **funcionalidad básica** y su **rendimiento** —idealmente los **niveles de servicio comprometidos (SLO)**— incluso cuando se enfrenta a situaciones estresantes o desafiantes.

La resiliencia es una propiedad esencial en sistemas distribuidos y en arquitecturas de microservicios, donde múltiples componentes o servicios trabajan juntos para brindar una funcionalidad completa. Estos sistemas están expuestos a una variedad de posibles fallos, como errores en los microservicios, problemas de red (latencias anómalas, particiones, caídas), sobrecargas, pérdidas de datos y otros eventos inesperados. La resiliencia busca garantizar que el sistema pueda **absorber el impacto**, **recuperarse** y **seguir funcionando** sin afectar negativamente a la experiencia del usuario, preferentemente mediante **degradación controlada** en lugar de interrupciones totales.

En términos prácticos, la resiliencia combina **diseño** y **operación**. En el diseño, se emplean patrones que limitan el alcance de las fallas y evitan su propagación: **timeouts y deadlines** para no bloquear recursos, **reintentos con exponential backoff + jitter** solo en operaciones **idempotentes**, **circuit breakers** para cortar llamadas hacia dependencias inestables, **aislamiento de recursos (bulkheads)** para que una dependencia lenta no “tire” el servicio completo, y **limitación/rechazo temprano de carga (rate limiting y load shedding)** para preservar la latencia de operaciones críticas. En lo operativo, la **observabilidad de**

tres señales (trazas, métricas y logs, correlacionadas por *trace-id*) permite **detectar antes, diagnosticar mejor y acortar el tiempo de recuperación (MTTR)**; los **health checks** (startup, liveness, readiness) y el *graceful shutdown/drain* evitan impactos durante despliegues y rotación de instancias.

La resiliencia también abarca la **consistencia y recuperación de datos**. En microservicios, muchas operaciones se resuelven con **consistencia eventual y coordinación por SAGA** (coreografía u orquestación) con pasos **compensatorios**. Para no perder mensajes en fallos transitorios, se usan **colas, reintentos controlados y Dead-Letter Queues (DLQ)**. A nivel de continuidad del negocio, se planifican **objetivos de recuperación** como **RTO** (tiempo) y **RPO** (punto de datos), con **redundancia** por zonas/regiones, **copias de seguridad verificadas** y **procedimientos de failover** ensayados.

Finalmente, la resiliencia es **medible y verificable**. Se gestiona con **SLI/SLO** y **error budgets** para tomar decisiones de cambio vs. estabilidad; se valida con **pruebas de resiliencia** periódicas (*game days*, *chaos testing* controlado) y con **capacidad elástica** que acompaña la demanda sin degradar la experiencia. Resiliencia no es lo mismo que “nunca falla”: es **fallar con gracia**, recuperarse rápido y **aprender** de cada incidente para reforzar el sistema.

Patrones y Buenas Prácticas en microservicios

Los Patrones y Buenas Prácticas en microservicios son un conjunto de **directrices, principios y estrategias que guían el diseño, desarrollo y despliegue efectivo de sistemas basados en microservicios**. Estas pautas están diseñadas para abordar los desafíos específicos que surgen al trabajar con arquitecturas de microservicios y ayudan a los equipos de desarrollo a construir sistemas más **robustos, escalables y fáciles de mantener**. En el contexto de los microservicios, los patrones y buenas prácticas abarcan la **comunicación entre servicios**, la **gestión de datos**, la **tolerancia a fallos**, la **seguridad**, el **despliegue** y la **escalabilidad**. Se han desarrollado y refinado a lo largo del tiempo a medida que crecía la adopción de esta arquitectura y resultan fundamentales para lograr un rendimiento y una confiabilidad consistentes en sistemas distribuidos.

Una de las principales preocupaciones en el diseño de microservicios es la **comunicación efectiva** entre ellos. Los patrones de **API Gateway** y **Agregación/Composición de APIs** ayudan a exponer una interfaz unificada y a reducir el *chattiness* entre cliente y servicios, mientras que el patrón **BFF (Backend for Frontend)** adapta esa interfaz a las necesidades de cada tipo de cliente sin

sobrecargar al gateway con lógica de negocio. Además de las llamadas **síncronas** (REST/gRPC), es recomendable incorporar **comunicación asíncrona** mediante **eventos** o **mensajería** para desacoplar temporalmente a los servicios. *Este enfoque se conoce como **Event-Driven Architecture (EDA)**, y puede implementarse con distintas tecnologías (por ejemplo, **colas de mensajería**, **brokers de eventos**, **WebSockets** o **Server-Sent Events**), favoreciendo la escalabilidad y la independencia temporal entre servicios.* En este plano son claves la **idempotencia**, la **deduplicación** y el **versionado de contratos** (OpenAPI/AsyncAPI) con estrategias de **compatibilidad hacia atrás** y el enfoque **tolerant reader** para evitar roturas ante cambios evolutivos.

En gestión de datos, la práctica de **database per service** garantiza que cada servicio controle su propio modelo y esquema, reduciendo acoplamientos ocultos. *Este patrón fomenta la **autonomía de datos** y evita dependencias directas a nivel de base de datos entre servicios.* Para coordinar operaciones que abarcan múltiples dominios se usan patrones como **SAGA** (por **coreografía** u **orquestración**) con pasos compensatorios, y **Transactional Outbox/CDC** para publicar eventos de manera confiable sin perder integridad. *En la **coreografía**, cada servicio reacciona a eventos de otros; en la **orquestración**, un servicio central coordina las acciones de todos los implicados.* Cuando la lectura requiere agregaciones entre dominios, **API Composition** o **CQRS** permiten construir vistas optimizadas sin introducir dependencias fuertes a nivel de almacenamiento, aceptando **consistencia eventual** donde corresponde.

La **resiliencia** se consigue combinando **diseño** y **operación**. A nivel de diseño, se aplican **timeouts** y **deadlines** en toda llamada remota, **reintentos** con *exponential backoff + jitter* únicamente sobre operaciones **idempotentes**, **circuit breakers** para aislar dependencias inestables, **bulkheads** para separar recursos y evitar fallas en cascada, y **rate limiting** con **load shedding** para proteger la latencia bajo saturación. Operativamente, se utilizan **health checks** (*startup, liveness, readiness*) y **graceful shutdown/drain** para desplegar y rotar instancias sin impacto perceptible. *Los **health checks** son un patrón clave para verificar la disponibilidad de cada servicio antes de enrutar tráfico hacia él.*

La **observabilidad** no es opcional en microservicios. Instrumentar **trazas**, **métricas** y **logs** con correlación por **trace-id** permite detectar antes, diagnosticar mejor y acortar el **MTTR**. Establecer **SLI/SLO** y **error budgets** ayuda a tomar decisiones de cambio vs. estabilidad; y validar la robustez con **pruebas de resiliencia** (*game days*, **chaos testing controlado**) asegura que los mecanismos de recuperación funcionen cuando realmente se los necesita.

En **despliegue** y **escalabilidad**, los patrones **Blue-Green** y **Canary** permiten introducir cambios de forma controlada, minimizar riesgo y validar comportamiento real con una fracción del tráfico. Se complementan con **rolling updates** y **feature flags** para habilitar **progressive delivery**. La **escalabilidad horizontal dirigida** exige **orquestadores de contenedores**, *mecanismos de Service Discovery* para la *resolución dinámica de endpoints* y *patrones de configuración centralizada como Spring Cloud Config*, así como **balanceo de carga** adecuados; y, cuando la plataforma lo justifica, el **sidecar/service mesh** aporta **mTLS**, **traffic shaping** y **retries** gestionados fuera del proceso de la aplicación, reduciendo la complejidad en el código.

La **seguridad** requiere un enfoque de **Zero Trust** con **autenticación** y **autorización** robustas entre clientes y servicios (**OAuth2/OIDC**) y entre servicios (**mTLS** y políticas de autorización), además de **gestión de secretos**, **validación de esquema** en el borde, **cifrado** en tránsito y en reposo y controles de **rate limiting** por identidad o tenant. Estas medidas reducen la superficie de ataque y ayudan a cumplir normativas sin frenar el desarrollo.

Es importante subrayar que los patrones y buenas prácticas en microservicios no son recetas universales. Son guías que deben adaptarse al **dominio**, a la **complejidad del sistema**, a los **requisitos no funcionales** y a la **madurez** del equipo y de la plataforma. La selección y el orden de adopción suelen ser graduales: comenzar por **límites correctos de dominio**, **datos propios** y **observabilidad**; continuar con **resiliencia** y **contratos evolucionables**; y, finalmente, introducir **progressive delivery**, **mesh** y automatizaciones avanzadas cuando el retorno lo justifique.

Circuit Breaker

El patrón de Circuit Breaker (Interruptor de circuito) es un patrón de diseño utilizado en arquitecturas de software para mejorar la resiliencia y la tolerancia a fallos en sistemas distribuidos, como los basados en microservicios. Su objetivo es proteger un sistema de colapsos en cadena causados por fallos en uno o varios servicios, evitando que el fallo se propague a otras partes del sistema.

El patrón se inspira en los interruptores de circuito eléctricos: ante sobrecarga o cortocircuito, **se abre** para cortar el flujo y evitar daños mayores. De forma análoga, en software el Circuit Breaker actúa como **barrera protectora** entre componentes, deteniendo llamadas hacia una dependencia que está fallando o respondiendo con latencias anómalas.

El funcionamiento clásico del patrón se divide en tres estados:

- **Estado Cerrado (Closed)**: En este estado, el Circuit Breaker permite que las solicitudes fluyan normalmente entre los componentes. Si las operaciones son exitosas, el breaker permanece cerrado. **Si aumenta la tasa de errores o de “llamadas lentas” (por ejemplo, por *timeouts*), el breaker acumula esos eventos en una ventana deslizante y evalúa umbrales configurados.**
- **Estado Abierto (Open)**: Cuando la tasa de fallos o de llamadas lentas alcanza un umbral predefinido, el breaker pasa a **abierto**. En este estado bloquea las solicitudes hacia la dependencia problemática y **falla rápido** (opcionalmente con un **fallback** seguro), durante un tiempo de espera configurado. Esto evita saturar a la dependencia y protege los recursos del llamador.
- **Estado Semiabierto (Half-Open)**: Tras el periodo abierto, el breaker pasa a **semiabierto** y **permite un número limitado de solicitudes de prueba** para evaluar la recuperación de la dependencia. Si una proporción suficiente de estas solicitudes resulta exitosa, **vuelve a cerrado**; si vuelven a fallar, **retorna a abierto** y reinicia el periodo de protección.

Detección y configuración práctica: Un Circuit Breaker moderno se configura con: **tamaño de ventana** (cantidad de llamadas recientes a observar), **umbral de fallo** (p. ej., 50–60 % de errores), **umbral de “llamadas lentas”** (p. ej., > X ms), **duración en abierto** (cuánto tiempo permanece abierto antes de probar), y **número/perfil de llamadas permitidas en semiabierto**. Es habitual **registrar qué excepciones cuentan como fallo** y cuáles deben **ignorarse** (p. ej., errores de validación 4xx).

Buenas prácticas de uso: Un Circuit Breaker **no sustituye** a los *timeouts*: siempre definir **timeouts** y **deadlines** en cada llamada remota. Combinar con **reintentos** solo para **errores transitorios** y **operaciones idempotentes**, usando **exponential backoff + jitter** para evitar tormentas de reintentos. Diseñar **fallbacks** que sean **seguros y explícitos** (degradación entendible para el usuario, datos cacheados, valores por defecto), y **no ocultar** indefinidamente una falla real. Aislar recursos con **bulkheads** y proteger la latencia con **rate limiting/load shedding**. Finalmente, **exponer métricas y eventos del breaker** (aperturas, cierres, tasa de errores, latencias) y **trazabilidad** para diagnóstico.

Diferencias con otros mecanismos.

- *Retry* reintenta; **Breaker bloquea** después de detectar inestabilidad.
- *Bulkhead* aísla recursos; **Breaker** corta el tráfico a la dependencia.
- *Rate limiting* controla entrada; **Breaker** reacciona al **comportamiento de la dependencia**.

Implementación típica: En stacks como Spring, es común usar **Resilience4j** / **Spring Cloud Circuit Breaker** para configurar ventanas, umbrales, duración en abierto, llamadas permitidas en semiabierto y fallbacks; integrarlo con **métricas/observabilidad** y con políticas de *retry* y *timeout* a nivel de cliente.

API Gateway

API Gateway es un patrón de diseño y un componente clave en arquitecturas de microservicios y sistemas distribuidos. Es una puerta de enlace (gateway) que actúa como punto de entrada único para las solicitudes de los clientes hacia los microservicios subyacentes. En otras palabras, el API Gateway proporciona una interfaz unificada y simplificada para que los clientes interactúen con el sistema en su conjunto, en lugar de comunicarse directamente con cada uno de los microservicios individualmente.

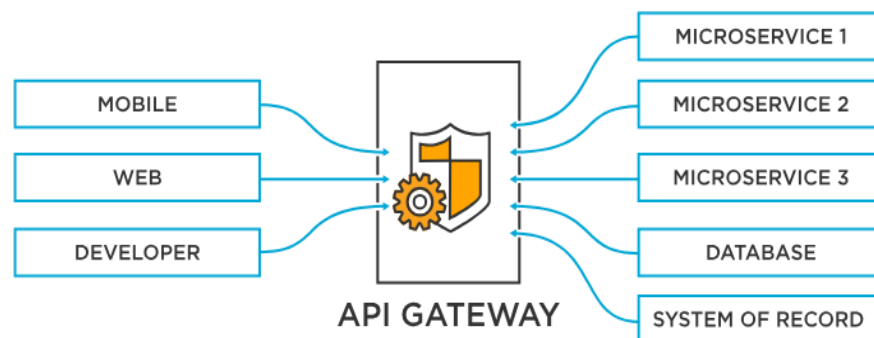


Imagen 4: Elaboración Propia.

Las principales funciones y características del API Gateway son las siguientes:

- **Unificación de servicios:** El API Gateway permite combinar múltiples microservicios y sus diferentes puntos de acceso en una única API pública. Los clientes pueden realizar todas sus solicitudes a través del API Gateway sin necesidad de conocer la estructura interna del sistema o interactuar con cada microservicio de manera individual.
- **Composición de solicitudes:** El API Gateway puede agrupar varias solicitudes de los clientes y realizar llamadas a diferentes microservicios para cumplir con la solicitud del cliente. Esto evita que el cliente realice múltiples llamadas a diferentes microservicios y permite que el API Gateway optimice la comunicación interna para mejorar el rendimiento.
- **Enrutamiento y redirección:** El API Gateway puede realizar el enrutamiento de las solicitudes entrantes hacia el microservicio adecuado según la lógica de negocio o los parámetros proporcionados por el cliente. También puede redirigir las solicitudes a diferentes versiones de los microservicios para permitir implementaciones de múltiples versiones.

- **Seguridad y autenticación:** El API Gateway actúa como un punto de control de acceso y puede gestionar la autenticación y autorización de los clientes antes de redirigir las solicitudes a los microservicios correspondientes. Esto centraliza la gestión de la seguridad y protege los microservicios de accesos no autorizados.
- **Caché de datos:** El API Gateway puede implementar una capa de **caché** para almacenar respuestas a solicitudes comunes y reducir la carga en los microservicios. Esto mejora la eficiencia y el rendimiento del sistema al reducir la cantidad de llamadas a los microservicios.
- **Monitoreo y análisis:** El API Gateway puede realizar el seguimiento y monitoreo de las solicitudes entrantes y salientes, lo que proporciona información valiosa para analizar el rendimiento del sistema, identificar posibles cuellos de botella y detectar problemas de manera temprana.
- **Rate limiting y cuotas:** Aplicar límites por API key/cliente/tenant, *burst control* y *throttling* para proteger la capacidad y evitar abusos.
- **Validación de esquema y seguridad en el borde:** Validar **OpenAPI/JSON Schema** o **AsyncAPI** en el gateway, filtrar/pulir *payloads* (p. ej., tamaño máximo), normalizar encabezados y aplicar políticas de **CORS** y **WAF**.
- **Autenticación y autorización modernas:** Soporte para **OAuth2/OIDC** (validación de **JWT**, *scopes/claims*), **API keys**, y **mTLS** entre clientes confiables y el borde. Rotación y revocación de credenciales centralizada.
- **Enrutamiento avanzado y *progressive delivery*:** *Canary*, *blue/green*, *A/B*, *shadow traffic* y **traffic splitting** por versión, peso o atributos del pedido.
- **Compatibilidad con *streaming*:** Soporte para **SSE** (cabeceras *text/event-stream*, deshabilitar *buffering* y *idle timeouts*), gRPC transcoding cuando aplique, y *websocket pass-through* si es necesario.
- **Resiliencia en el borde:** **Timeouts**, **circuit breaking**, **reintentos con backoff + jitter** solo cuando sea seguro, límites de tamaño/tiempo de petición y **idempotency keys** para operaciones sensibles.
- **Observabilidad de tres señales:** Métricas (RPS, tasa de errores, P95/P99, saturación), **logs estructurados** y **trazas** con propagación de **W3C Trace Context**; inclusión de *traceld* en respuestas de error para diagnóstico.

- **Transformación/normalización:** *Request/response mapping* (por ejemplo, agregar/quitar encabezados, normalizar formatos), *response compression* y negociación de contenido.
- **Integración con *service discovery* y balanceo:** Resolución dinámica de *backends* (Kubernetes/Consul/Eureka/DNS), **health checks** y **balanceo L7** con *outlier detection*.
- **Alta disponibilidad y escalado:** Desplegar el gateway en **modo redundante**, *stateless* donde sea posible, con **escalado horizontal** y, si aplica, **multi-región** para evitar un punto único de falla.
- **Límite funcional del gateway (antipatrones):** Mantener el gateway **delgado**. La **lógica de negocio** y orquestación compleja deben vivir en los servicios o en un **BFF** por cliente; el gateway **no** debe convertirse en un “monolito en la puerta”.

El API Gateway no solo beneficia a los clientes que interactúan con el sistema, sino también a los desarrolladores y equipos de operaciones. Proporciona una capa de abstracción que facilita la gestión, el mantenimiento y la evolución de la arquitectura de microservicios. Además, el API Gateway puede ser implementado como un servicio independiente o como parte de la plataforma de entrada, **pero sin mezclarse con orquestación de negocio**.

Es importante que su diseño e implementación consideren los requisitos específicos del sistema y la **escalabilidad/alta disponibilidad** necesarias para gestionar la carga de solicitudes entrantes. Una selección cuidadosa de herramientas y una correcta configuración de políticas son fundamentales para garantizar un funcionamiento eficiente y confiable del API Gateway en el contexto de una arquitectura de microservicios.

Health Check

En un entorno de microservicios, no basta con que un proceso esté “encendido”, teniendo en cuenta también que puede haber varias instancias de un mismo microservicio repartido en diferentes servidores en donde no todas las instancias tienen el mismo dominio.

- Un servicio puede estar corriendo, pero **no responder correctamente**.
- Otros servicios necesitan saber si es seguro enviarle peticiones.
- Orquestadores como **Kubernetes** o balanceadores de carga dependen de esta información.

Health Check permite verificar si un servicio está sano y disponible. Cada servicio expone un **endpoint especial de salud**.

- Devuelve un mensaje indicando su estado actual.
- Puede incluir detalles: conexión a base de datos, colas, dependencias externas.
- El orquestador usa esa información para decidir si enviar tráfico o reiniciar el contenedor.

Un ejemplo claro de este comportamiento es el endpoint **/ping** que normalmente tenemos en nuestros servicios que indica que nuestro servicio ya está listo para recibir peticiones.

Rate Limit

En sistemas distribuidos expuestos a internet, hay riesgos:

- Un cliente puede enviar **demasiadas peticiones** y saturar el sistema.
- Servicios internos pueden consumir recursos en exceso.
- Se necesitan **garantías de SLA** (ej. 100 req/s por usuario).

El **Rate Limit**: restringir la cantidad de peticiones permitidas en un período de tiempo. Se usa un **algoritmo de control de tráfico** que define:

- Límite (ej. 10 requests por segundo).
- Ventana temporal (ej. 1 min).
- Acción si se excede: rechazar (429 Too Many Requests) o poner en cola.
- **Fixed Window / Sliding Window**: cuentan peticiones en intervalos de tiempo.

BFF (Backend for Frontend)

El **BFF (Backend for Frontend)** es un **patrón de diseño** y una **capa intermedia** dedicada que se ubica entre los **clientes** (web, móvil, paneles internos) y los **microservicios de dominio**. Su finalidad es **adaptar y componer** respuestas a medida de cada tipo de frontend, sin sobrecargar al **API Gateway** ni duplicar lógica en las aplicaciones cliente. En términos prácticos, el BFF entrega **payloads listos para renderizar**, reduce **viajes de red** y estabiliza **contratos específicos por canal**.

Un ejemplo cotidiano puede ser la app de delivery como PedidosYa. El **usuario** quiere ver menús y tiempos de entrega, el **restaurante** necesita gestionar pedidos con datos del cliente y el **repartidor** solo requiere direcciones y rutas. Si todos consumieran directamente los mismos microservicios, deberían procesar información innecesaria y hacer múltiples llamadas. Con un BFF, en cambio, cada canal recibe solo los datos que necesita en el formato adecuado, mejorando la eficiencia y la experiencia de uso.

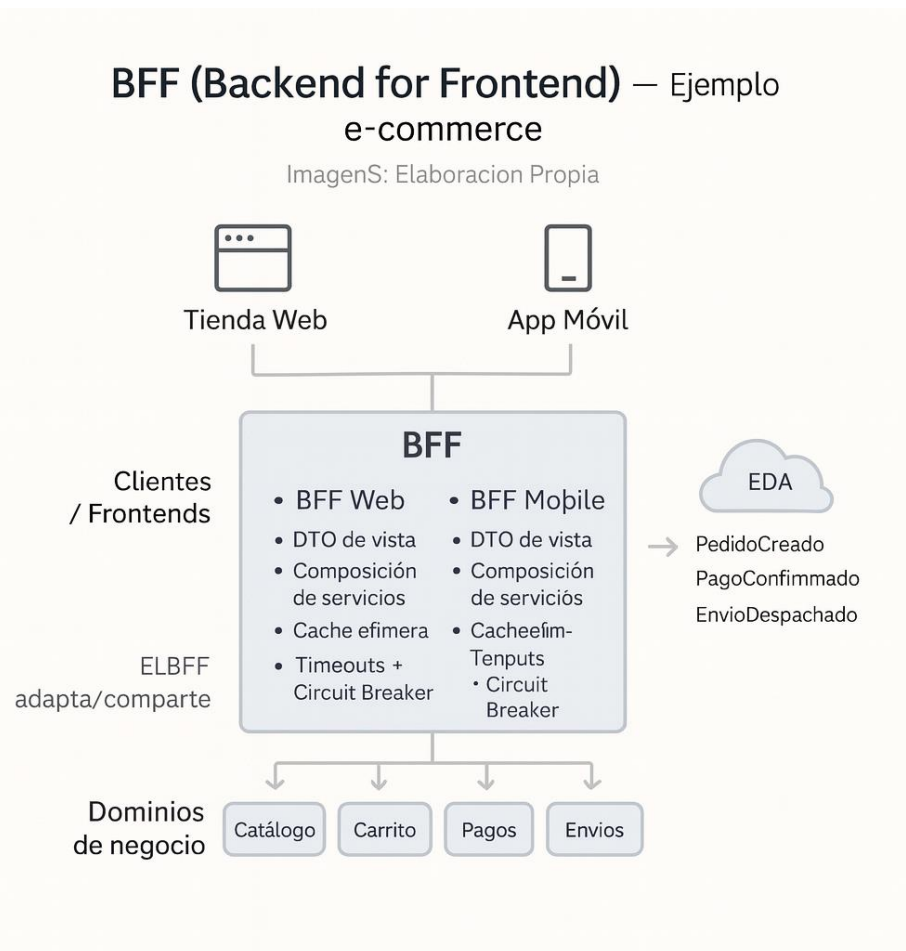


Imagen 5: Elaboración Propia.

Funciones y características principales del BFF

- **Interfaz a medida por cliente:** define **endpoints** y **modelos específicos** para cada **canal** (p. ej., **/bff/web/**, **/bff/mobile/**), entregando exactamente los datos que la UI necesita en la **granularidad** adecuada.
- **Composición y agregación de datos:** realiza **fan-out** hacia múltiples **microservicios**, **une** y **normaliza** la información en **una única respuesta** coherente para la pantalla; la **lógica de negocio** permanece en los **servicios de dominio**.
- **Transformación/normalización de modelos:** **mapea** campos, **oculta** atributos **innecesarios** o **sensibles**, unifica **formatos** de **fecha/moneda/idioma** y aplica **paginación/proyecciones** alineadas al uso real.
- **Optimización de latencia y chattiness:** reduce **N llamadas** del cliente a **1 solicitud** al BFF, ejecuta consultas en **paralelo** y puede aplicar **caché efímera** (de muy corto plazo) para vistas costosas.
- **Seguridad por vista:** aplica **scopes/claims mínimos** por pantalla, **filtra** datos **sensibles** y registra **logs estructurados** sin información confidencial.
- **Contratos estables y versionados:** publica **OpenAPI por canal**, favoreciendo la **compatibilidad hacia atrás** y la **evolución independiente** de cada UI.
- **Resiliencia entre capas:** usa **timeouts**, **circuit breakers**, **bulkheads** y **reintentos** solo cuando es **seguro** (operaciones **idempotentes**); devuelve errores claros con **identificadores de correlación**.
- **Observabilidad orientada a la experiencia:** expone **métricas** (RPS, **P95/P99** por endpoint), **trazas** con **propagación de contexto** y **logs** que permiten diagnosticar **extremo a extremo** la interacción de cada vista.
- **Compatibilidad con actualizaciones orientadas a eventos:** facilita que las UIs reciban cambios en **tiempo casi real** mediante un canal **event-driven**, sin acoplar la UI a los servicios de dominio.

Posicionamiento en la arquitectura

Cliente → **API Gateway** → **BFF específico** → **Microservicios**
(políticas transversales) **por canal** **de dominio.**

El Gateway centraliza enrutamiento, autenticación, rate limiting y validaciones; el BFF adapta y compone la respuesta para cada experiencia de usuario.

Límite funcional del BFF (antipatrones)

- No debe convertirse en un “**mini-monolito**” compartido por todos los clientes: preferir **un BFF por tipo de canal**.
- No debe implementar **reglas de negocio de dominio** ni **persistir datos de negocio**; su foco es **presentación y composición**.
- No debe exponer **modelos internos** de los servicios; publicar **DTOs de vista** específicos y **estables**.

El BFF **complementa** al **API Gateway** al proporcionar **contratos específicos**, **respuestas compuestas y optimizadas** y controles de **seguridad/observabilidad** centrados en la experiencia. Esta **separación de responsabilidades** acelera la **evolución** de las UIs, reduce **complejidad** en el cliente y mantiene **delgado** el gateway, preservando la **cohesión del dominio** en los microservicios.

Event-Driven Architecture (EDA)

La **Event-Driven Architecture (EDA)** es un **estilo arquitectónico** en el que los **eventos** —hechos de negocio ya ocurridos— son la unidad principal de **comunicación asíncrona** entre componentes. Los **productores** publican eventos y los **consumidores** reaccionan **sin acoplamiento directo**, favoreciendo **escalabilidad, resiliencia y evolución independiente**.

Cuando un usuario confirma la compra, se publica un **evento** “OrdenCreada”. Ese evento es **consumido por distintos servicios**: el de pagos inicia la transacción, el de inventario descuenta el stock, el de envíos genera la guía y el de notificaciones envía un correo al cliente. Ninguno de estos servicios depende de los otros ni conoce sus detalles, simplemente reaccionan al evento. Así, el sistema puede escalar y evolucionar de manera independiente, mientras mantiene una coordinación basada en hechos ya ocurridos.

Event-Driven Architecture (EDA) — Visión mínima

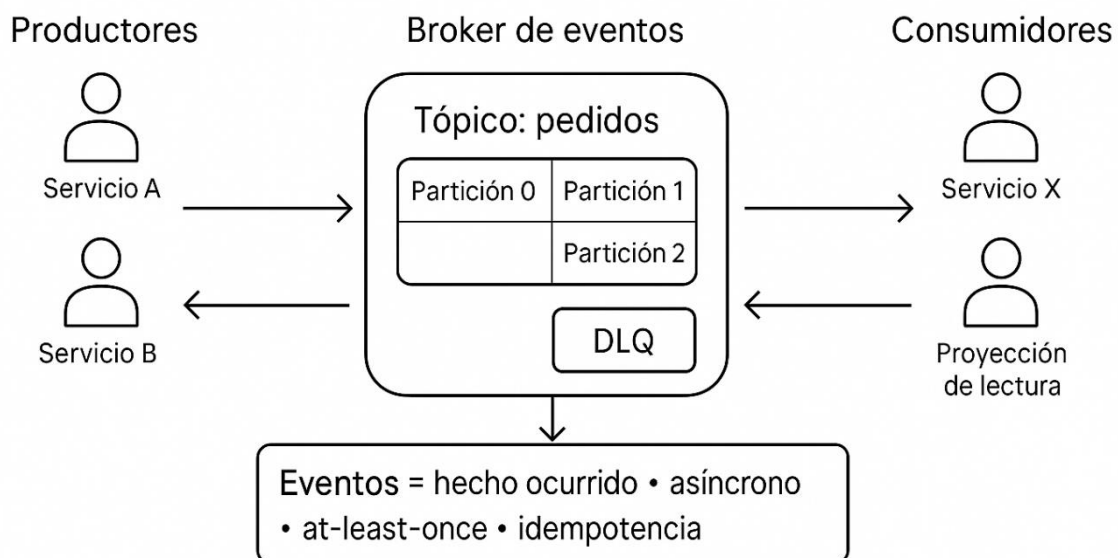


Imagen 6: Elaboración Propia.

Las principales funciones y características de la EDA son las siguientes:

- **Desacoplamiento temporal y espacial:** productores y consumidores evolucionan y **escalan por separado**; agregar consumidores no exige cambios en productores.
- **Asincronía y amortiguación de picos:** un **broker/stream** (tópicos/colas, **retención**) actúa como **búfer**, permite **procesamiento diferido** y **reintentos** sin bloquear.

- **Contratos de eventos:** esquemas explícitos (JSON Schema/Avro/Proto), **versionado** compatible y **metadatos** (timestamp, aggregateId, correlationId/causationId).
- **Semántica de entrega e idempotencia:** diseño para **at-least-once** con **consumidores idempotentes** y **deduplicación** cuando aplique.
- **Orden y particionamiento:** **claves de partición** para preservar el **orden por agregado** (p. ej., **orderId**) y habilitar **paralelismo** seguro.
- **Gestión de errores robusta:** **reintentos** con **backoff + jitter** y **dead-letter queue (DLQ)** para aislar mensajes problemáticos.
- **Consistencia eventual consciente:** aceptar **propagación diferida**; para acoplar **DB y evento**, usar **Transactional Outbox/CDC**.
- **Proyecciones y materialized views:** construir **vistas de lectura** rápidas a partir del **log de eventos**.
- **Observabilidad y gobernanza:** **métricas** de **lag/throughput/error**, **trazabilidad** con **trace-id** y **políticas de retención/permiso** por tópico.

Límite funcional de la EDA (antipatrones)

- Evitar **RPC sobre eventos** (request/response apretado) que rompan el **desacoplamiento**.
- No usar eventos como **sustituto de toda consulta**; preferir **proyecciones de lectura**.
- No depender de **exactly once**; diseñar **idempotencia** y **reprocesamiento** seguro.

EDA coloca a los **eventos** en el centro para lograr **reactividad**, **escala** y **robustez**, a cambio de **disciplina** en **contratos**, **idempotencia**, **orden/particionamiento**, **manejo de errores** y **observabilidad**.

Service Discovery

En una arquitectura monolítica, todas las funciones del sistema viven en un único bloque de código o instancia de un servicio. Las clases se comunican entre sí directamente en memoria, por lo que **“descubrir” un servicio significa simplemente invocar un método de otra clase.**

Sin embargo, en una arquitectura de **microservicios**, cada servicio vive en su propio proceso, usualmente desplegado en contenedores o máquinas distintas.

Imaginemos que vas a un hospital grande y necesitás ver a un especialista. En lugar de recorrer pasillos buscando en qué consultorio está, vas a recepción y allí te dicen: “El cardiólogo está en el consultorio 305, el traumatólogo en el 210”. Esa **recepción** funciona como un **Service Discovery**: un punto central que sabe dónde está cada médico (servicio) en ese momento. Si un consultorio se muda de piso, vos no necesitás enterarte ni memorizar el cambio; solo volvés a preguntar en recepción y obtener la nueva ubicación.

¿Cómo encuentra un servicio al otro?

- En un sistema pequeño, podríamos codificar manualmente las direcciones IP y puertos de cada servicio.
- Pero en sistemas dinámicos (Kubernetes, Docker, nubes públicas), las direcciones cambian constantemente.
- Sin un mecanismo confiable de descubrimiento, el sistema se rompe: un **catálogo de productos** no puede consultar al **servicio de pagos**, o el **servicio de usuarios** no logra autenticar clientes.

Service Discovery surge para resolver este desafío: un mecanismo automático que permite a los servicios **encontrarse** y **comunicarse** sin depender de **direcciones fijas**. Siendo el proceso mediante el cual un servicio localiza la ubicación de otro servicio dentro de una red distribuida, sin depender de configuraciones manuales de IP o puertos.

Tenemos dos estrategias para el descubrimiento de servicios:

- **Client-Side Discovery**: El cliente consulta directamente al registro de servicios (ej: Eureka, Consul) para saber dónde está el servicio destino.
 - **Pros**: control total desde el cliente.
 - **Contras**: lógica de descubrimiento repetida en muchos clientes.

- **Server-Side Discovery:** El cliente hace la petición a un *load balancer* o *gateway*, que consulta al registro de servicios y reenvía la petición.
 - **Pros:** centralización de la lógica.
 - **Contras:** dependencia en un intermediario.

El patrón está compuesto principalmente por tres partes:

1. **Registro de servicios (Service Registry)** una base dinámica donde cada servicio se “anota” al arrancar y se “borra” al apagarse.
2. **Clientes de descubrimiento** librerías o frameworks que permiten a los servicios consultar el registro.
3. **Mecanismos de salud** (health checks) el registro elimina de su lista a los servicios que están caídos.

Database Per Service

En una arquitectura monolítica, todos los módulos comparten la **misma base de datos**. Esto permite hacer consultas complejas con *joins* entre tablas, pero genera un **fuerte acoplamiento**:

- Un cambio en el esquema puede romper a otros módulos.
- Es difícil escalar si todos dependen del mismo motor de base de datos.
- Se pierde autonomía en los equipos de desarrollo.

En microservicios, cada servicio debe ser **dueño exclusivo de sus datos**. Esto se conoce como **Database per Service**. Las características de este patrón son:

- Cada microservicio tiene su propia base de datos física o lógica.
- Nadie accede directamente a la base de otro servicio.
- La única forma de consultar o modificar datos de otro servicio es mediante su **API o eventos publicados**.

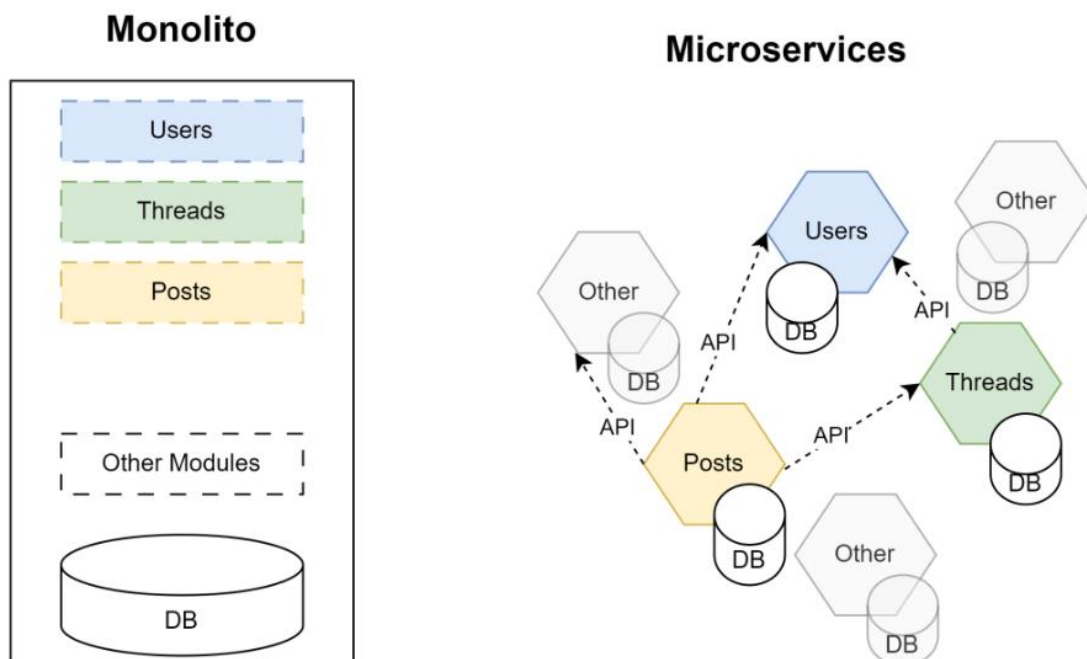


Imagen 7: Elaboración Propia.

SAGA

En sistemas monolíticos, las transacciones son fáciles: una operación puede actualizar varias tablas en la misma base de datos con garantías **ACID** (Atomicidad, Consistencia, Aislamiento, Durabilidad).

Pero en microservicios con **Database per Service**, cada servicio tiene su propia base de datos. Esto genera un problema:

- ¿Cómo aseguramos consistencia si una operación abarca **varios servicios**?
- Ejemplo: un pedido requiere actualizar **pagos, inventario y envíos**.

No podemos usar una transacción distribuida clásica porque introduce:

- Fuerte acoplamiento.
- Complejidad operativa.
- Baja tolerancia a fallos en sistemas distribuidos.

Imaginemos que organizas un viaje con amigos. Primero reservar el hotel, luego compras los pasajes y finalmente contratás una excursión. Cada paso es independiente y tiene sus propias reglas (el hotel confirma la reserva, la aerolínea emite el pasaje, la agencia agenda la excursión). Ahora bien, si al final falla la reserva del hotel, no tiene sentido mantener los pasajes o la excursión: necesitás cancelarlos. Ese proceso de **ir avanzando con pasos locales y, en caso de error, deshacer los previos** se parece a una **SAGA**. En algunos casos, vos mismo coordinás todo (orquestración), decidiendo qué cancelar o confirmar; en otros, cada proveedor reacciona a lo que ocurre (coreografía), como cuando la agencia devuelve el dinero de la excursión automáticamente al enterarse de que el hotel no se confirmó.

Aquí entra **SAGA**: un patrón para manejar **transacciones distribuidas** mediante **una secuencia de pasos locales y compensaciones**.

SAGA es una transacción larga compuesta por:

- **Pasos locales**: operaciones ACID dentro de un microservicio.
- **Acciones compensatorias**: operaciones que revierten el efecto de un paso previo en caso de fallo.

En lugar de un gran commit global, la consistencia se logra mediante **consistencia eventual**. Se compone principalmente de:

- **Servicios participantes**
 - Cada uno ejecuta su transacción local y, si falla el flujo, ejecuta una compensación.
- **Coordinación**
 - **Orquestada**: un servicio central (orchestrator) controla el flujo.
 - **Coreografiada**: no hay servicio central; cada microservicio reacciona a eventos de otros.
- **Acciones compensatorias**
 - Deben estar bien definidas.
 - Ejemplo: si se debita un pago y luego falla el inventario, se debe revertir el pago.

Modos de SAGA

Orquestación

En el **modo de orquestación**, la coordinación recae en un servicio central llamado **orquestador**. Este servicio actúa como “director de la orquesta”: envía comandos a los distintos microservicios para que realicen sus tareas en un orden específico.

Si en algún punto del proceso ocurre un fallo, el orquestador es el encargado de decidir qué acciones de **compensación** se deben ejecutar para revertir o mitigar el efecto del error.

- **Ventaja**: el flujo es **controlado y explícito**. Es fácil seguir el camino de la transacción, ya que todo está centralizado en un único servicio.
- **Desventaja**: se introduce un **punto único de coordinación**, lo que puede convertirse en un **cuello de botella** y, en el peor de los casos, en un punto único de fallo.

Este enfoque suele ser más sencillo de comprender y de mantener al inicio, pero puede limitar la escalabilidad y la autonomía de los microservicios si no se diseña cuidadosamente.

Coreografía

En el **modo de coreografía**, no existe un coordinador central. En su lugar, cada microservicio participa en la saga a través de **eventos**: cuando un servicio completa su tarea, publica un evento, y otros servicios interesados reaccionan a dicho evento para continuar con la transacción.

Este modelo funciona como un “baile coordinado” en el que cada participante sabe qué hacer cuando escucha una determinada señal. La lógica del flujo se encuentra distribuida entre los servicios en lugar de centralizarse.

- **Ventaja:** es un enfoque más **descentralizado y escalable**, ya que no depende de un único servicio para coordinar todo el proceso.
- **Desventaja:** el flujo de la saga es **implícito y disperso**, lo que lo hace más difícil de seguir, comprender y depurar.

La coreografía resulta atractiva en sistemas altamente distribuidos y dinámicos, pero requiere una disciplina rigurosa en el diseño de eventos y en la observabilidad del sistema para evitar comportamientos inesperados.

SIDECAR

En microservicios, los **cross-cutting concerns** (logs, métricas, TLS/mTLS, retries, tracing, configuración dinámica) tienden a duplicarse si cada equipo los resuelve dentro del servicio. **Sidecar** externaliza esas capacidades en un **proceso/contenedor adjunto** que comparte red/volúmenes/ciclo de vida con el servicio, manteniendo el foco del microservicio en su dominio. En Kubernetes, “sidecar container” es una primitiva estable que corre **junto al contenedor principal** en el mismo Pod y puede compartir red y volúmenes.

Un **Pod** en Kubernetes es la unidad mínima de despliegue que agrupa uno o más contenedores, los cuales comparten red, almacenamiento y ciclo de vida.

Sidecar es un patrón de despliegue donde un proceso (o contenedor) auxiliar se acopla al servicio para **extenderlo y estandarizar** capacidades operativas, compartiendo ciclo de vida. Se modela como un patrón “single-node”: dos contenedores co-agendados que comparten red/hostname/FS y donde el sidecar “aumenta” al contenedor de aplicación, a menudo **sin que éste lo note**.

Un ejemplo común sería que el televisor como servicio principal y control remoto como el sidecar siendo que no funciona solo, está siempre “pegado” a la experiencia del televisor y le agrega funcionalidad al televisor.

Beneficios clave

- **No intrusivo:** políticas comunes (mTLS, retries, timeouts, tracing) **sin tocar** el código del servicio.
- **Estandarización y poliglotismo:** un mismo sidecar sirve a servicios Java, Node, Go, etc.
- **Evolución independiente:** actualizás capacidades operativas sin redeploy funcional.

Compromisos

- **Overhead por Pod** (CPU/RAM) y mayor número de procesos a operar/monitorizar.

El sidecar soluciona problemas comunes como la observabilidad mediante prometheus y grafana. También existe Envoy soportando retries, retry budgets y circuit breaking configurables a nivel de cluster/upstream.

Domain-Driven Design (DDD)

Ofrece un marco conceptual para **alinear software con el negocio** y definir microservicios con límites claros. Propone diseñar el sistema basado en el **dominio del negocio** y no en aspectos técnicos. Esto significa que:

- Cada servicio debe representar un **bounded context** (contexto delimitado).
- El modelo de datos y reglas de negocio de un servicio **no deben filtrarse ni mezclarse** con otros.
- La comunicación entre servicios debe reflejar **lenguajes ubicuos** (terminología compartida entre negocio y desarrollo).

Cuando diseñamos microservicios, la primera gran pregunta es: **¿Cómo dividir el sistema en servicios?**

Si se hace mal, aparecen problemas clásicos:

- **Nano-servicios:** servicios demasiado pequeños que generan más sobrecarga que valor.
- **God services:** servicios gigantes que terminan siendo “monolitos distribuidos”.
- **Acoplamiento innecesario:** servicios que dependen fuertemente de otros.

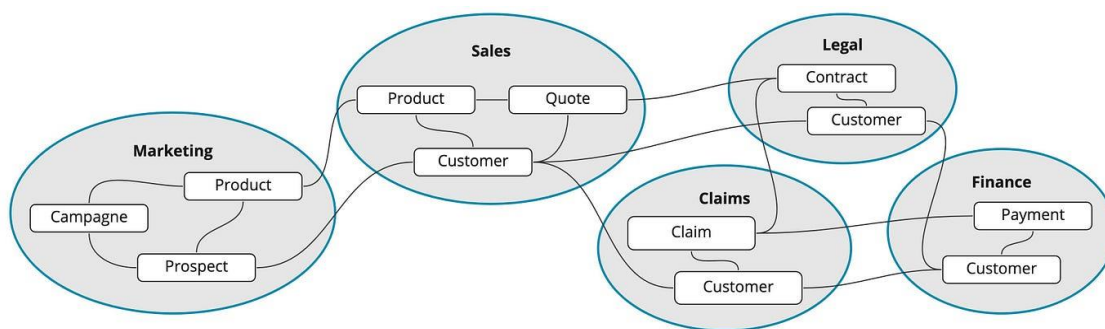


Imagen 8: Elaboración Propia.

Las partes principales del DDD son dominio, lenguaje ubicuo, bounded context y context mapping.

Domain (dominio)

El **dominio** es todo aquello que engloba el problema que queremos resolver. Si pensamos en un sistema de comercio electrónico, el dominio incluye ventas, pagos, logística, devoluciones, etc.

Dentro de un dominio amplio, es común identificar **subdominios** más específicos. Por ejemplo:

- En **ventas**: gestión de pedidos, facturación, promociones.
- En **logística**: seguimiento de envíos, asignación de transportistas, gestión de almacenes.

Los subdominios ayudan a dividir un sistema grande en partes manejables, cada una con su propio vocabulario, reglas y objetivos.

Lenguaje Ubicuo

Uno de los aportes más valiosos de DDD es el concepto de **lenguaje ubicuo**. Significa que todo el equipo (desarrolladores, analistas, expertos de negocio) comparte un mismo conjunto de términos para describir procesos, entidades y reglas dentro de un contexto determinado.

Este lenguaje se utiliza en las reuniones, en la documentación y en el código. Por ejemplo, si el negocio habla de “autorizar un pago” o “cancelar un envío”, esos mismos términos deben aparecer en las clases, métodos y servicios del software.

El objetivo es evitar ambigüedades. No es lo mismo “cliente” en el contexto de *ventas* (persona que compra) que en el contexto de *soporte* (persona que reclama). Por eso, el lenguaje debe estar **atado al contexto**.

Bounded Context

El **bounded context** es uno de los conceptos centrales de DDD. Se refiere al límite dentro del cual un modelo de dominio es válido y consistente.

En la práctica, significa que un mismo término puede tener significados distintos en contextos diferentes, y no pasa nada: lo importante es que dentro de cada bounded context no haya confusión.

Context Mapping

En sistemas grandes siempre habrá **múltiples bounded contexts**. La siguiente pregunta es: ¿cómo se relacionan entre sí?

Aquí entra el **context mapping**, que describe los distintos tipos de relaciones posibles:

- **Partnership**: dos contextos colaboran en igualdad, evolucionan juntos.
- **Customer–Supplier**: uno depende del otro (ej. un servicio de órdenes depende de un servicio de pagos).
- **Shared Kernel**: comparten una pequeña parte del modelo, pero implica riesgo de acoplamiento.
- **Anti-Corruption Layer (ACL)**: un contexto introduce una capa de traducción para protegerse de las complejidades o inconsistencias de otro.

En un supermercado grande, el **dominio** completo es el negocio en su conjunto: ventas, logística, atención al cliente y proveedores. Dentro de ese dominio aparecen **subdominios** más específicos: en ventas se gestionan pedidos y promociones, en logística se manejan depósitos y transporte, y en atención al cliente se resuelven reclamos. Cada sector utiliza su propio **lenguaje ubicuo**: para ventas, un “cliente” es quien compra; para reclamos, el mismo término puede referirse a quien presenta una queja. Para evitar confusiones, cada área funciona como un **bounded context**, donde los conceptos y reglas tienen un significado claro y limitado. Finalmente, cuando estos sectores necesitan interactuar (por ejemplo, logística informando a ventas que un producto está agotado), se aplica un **context mapping** que define cómo se comunican sin que cada uno pierda la coherencia de su propio modelo.

Load Balancer

En un sistema de software moderno, especialmente en arquitecturas basadas en microservicios, los usuarios pueden realizar **miles o millones de solicitudes simultáneas**. Si todas esas solicitudes se dirigieran a un único servidor, ocurrirían problemas evidentes:

- Sobrecarga del servidor.
- Caídas del sistema ante picos de tráfico.
- Desigualdad en la distribución de recursos.

El load balancer actúa como un “**director de orquesta**”: recibe todas las peticiones entrantes y las reparte entre diferentes servidores (instancias de aplicación, microservicios o nodos).

Si lo aplicamos a la vida cotidiana sería como la fila de cajas en un supermercado, donde un empleado organiza a los clientes y los dirige a las cajas desocupadas.

Si no se utiliza un balanceador de carga, se cae en problemas como:

- **Single Point of Failure**: si un servidor falla, toda la aplicación queda inoperante.
- **Ineficiencia**: algunos servidores están saturados mientras otros están ociosos.
- **Escalabilidad limitada**: difícil agregar o quitar recursos dinámicamente.

Estrategias de balanceo comunes

- **Round Robin**: distribuye las peticiones en orden secuencial.
- **Least Connections**: envía la nueva petición al servidor con menos conexiones activas.
- **IP Hash**: utiliza la IP del cliente para determinar siempre el mismo servidor.

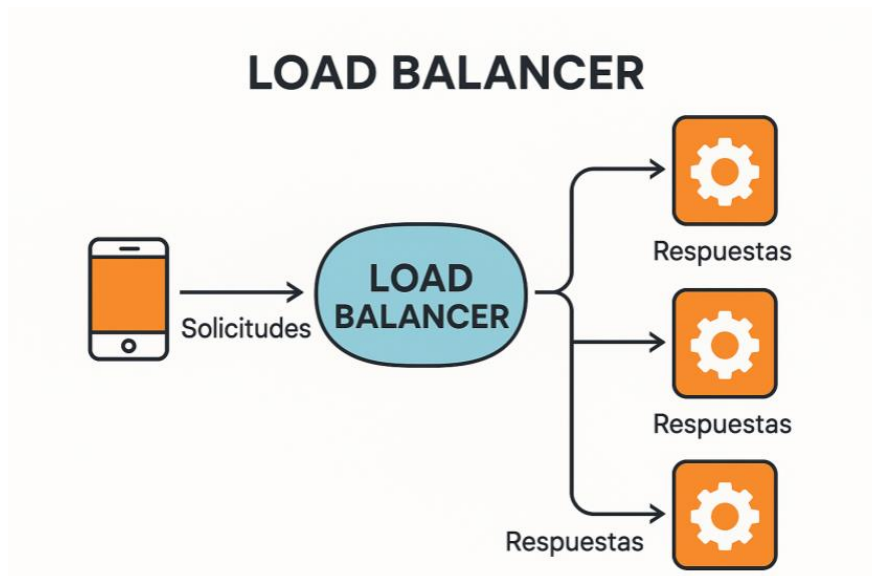


Imagen 9: Elaboración Propia.



Atribución-No Comercial-Sin Derivadas

Se permite descargar esta obra y compartirla, siempre y cuando no sea modificado y/o alterado su contenido, ni se comercialice. Universidad Tecnológica Nacional Facultad Regional Córdoba (S/D). Material para la Tecnicatura Universitaria en Programación, modalidad virtual, Córdoba, Argentina.